Chitkara

PAPER OF PT5-ACC-24CAI0104-B2024-SEM2-CLONE EXAM

Q1. Add two times.

Complete the code to add two times using class.

The input consists of 6 lines, with first three lines representing hours, minute and seconds of time1 and next three lines representing hour, minute and seconds of time2.

Output Format:

The output prints three lines.

First line gives time1 as – [hour:minute:seconds]

Second line gives time2 as – [hour:minute:seconds]

Third line gives resultant time as – [hour:minute:seconds]

Please note there is space after opening bracket, before closing bracket and before a new line.

Code Constraints:

In this scenario, the test cases fall under the following constraints:

$1 \leq N \leq 150$

Sample Testcase 1

Input:

2 //hour

30// minute

50 //second

5 //hour

45 //minute

30// second

```
Output:

[ 2:30:50 ] // first time input

[ 5:45:30 ] // second time input

[ 8:16:20 ] // sum of two times
```

ANSWER:

Test case 1 :

Input:
2
30
50
5
45
30

Output:
[2:30:50]
[5:45:30]
[8:16:20]

Test case 2 :

Input:
2
15
53
3
51
20

Output:
[2:15:53]
[3:51:20]
[6:7:13]

Test case 3 :

Input:
2

15

53

5

45

20

Output:

[2:15:53]
[5:45:20]
[8:1:13]

Test case 4 :

Input:
1

20

15

6

55

36

Output:
[1:20:15]
[6:55:36]
[8:15:51]

Test case 5 :

Input:
5

20

35

6

55

56

Output:
[5:20:35]
[6:55:56]
[12:16:31]

Q2. Decimal to binary

Given a decimal number as input, we need to write a program to convert the given decimal number into an equivalent binary number.

Sample Input:

34

Sample Output:

100010

Note: The body of main function is already provided in tail part.

ANSWER:

Test case 1 :

Input:
34

Output:
100010

Test case 2 :

Input:
343

Output:
101010111

Test case 3 :

Input:
15002

Output:
11101010011010

Q3. Safe Class

One of the weakness of C++ is that it does not automatically check array indexes to see whether they are in bounds. (This makes array operations faster but less safe). We can use a class to create a safe array that checks the index of all array accesses.

Write a class called SafeArray that uses an int array of fixed size (say LIMIT) as its only data member, Fix it to 20 for this program. There will be two member functions.

- The first, putel(), takes an index number and an int value as arguments and inserts the int value into the array at the index.
- The second, getel(), takes an index number as an argument and returns the int value of the element with that index.

```
SafeArray sa1;          // define a safearay object
int temp = 125;         // define an int value
sa1.putel(7, temp);             // insert value of temp into array at index 7
temp = sa1.getel(7);    // obtain value from array at index 7
```

Both functions should check the index argument to make sure it is not less than 0 or greater than LIMIT-1. Otherwise that element will not be printed and a message "Out of Bounds" will be printed instead.

🡅Sample Input

```
2
125
7
135
31
```

Sample Output

```
125
Out of Bounds
```

Explanation

First line of input denotes the number of elements. Next each two lines denotes the number and its index in array.

Print the number in new line after inserting it in the array.

ANSWER:

Test case 1 :

Input:
2
125
7
135
31
Output:
125
Out of Bounds

Test case 2 :

Input:
5
10 0
20 15
30 19

40 25
50 100

Output:
10
20
30
Out of Bounds
Out of Bounds

Test case 3 :

Input:
4
25
4
15
1
2
2
555
9
Output:
25
15
2
555

Test case 4 :

Input:
2
125
7
135
31

Output:
125
Out of Bounds

Test case 5 :

Input:
2
125
7
135
1
Output:
125
135

Test case 6 :

Input:
1

999 0

Output:
999

Q4. Strong Numbers

A strong number is a number whose sum of the factorials of its digits is equal to the number itself. For example, 145 is a strong number because 1!+4!+5!=1451! + 4! + 5! = 145. Write a program in C++ to find all the strong numbers between two given numbers `a` and `b` (inclusive).

Input Format:

- The first line contains two integers `a` and `b` (where `a <= b`).

Output Format:

- Print all strong numbers between `a` and `b` in ascending order.
- If no strong numbers exist, print `No strong numbers found`.

Constraints:

1. 1≤a≤b≤10^6
2. The program should calculate factorials for digits only (i.e., from 0 to 9).

Sample Input 1:

```
1 200
```

Sample Output 1:

```
1 2 145
```

Sample Input 2:

```
300 400
```

Sample Output 2:

```
No strong numbers found
```

Explanation:

1. In the first example, the strong numbers between 1 and 200 are 1, 2, and 145.

- For 1: 1!=11! = 1
- For 2: 2!=22! = 2

- For 145: 1!+4!+5!=1+24+120=1451! + 4! + 5! = 1 + 24 + 120 = 145

1. In the second example, there are no strong numbers between 300 and 400.

ANSWER:

Test case 1 :

Input:
1 200

Output:
The Strong numbers in the given range are: 1 2 145

Test case 2 :

Input:
300 400
Output:
The Strong numbers in the given range are: No strong numbers found

Test case 3 :

Input:
1 10

Output:
The Strong numbers in the given range are: 1 2

Test case 4 :

Input:
100 500
Output:
The Strong numbers in the given range are: 145

Test case 5 :

Input:
200 500
Output:
The Strong numbers in the given range are: No strong numbers found

Test case 6 :

Input:
10 1000
Output:
The Strong numbers in the given range are: 145

Test case 7 :

Input:
1 2000000

Output:
The Strong numbers in the given range are: 1 2 145 40585

Q5. Student Score

Develop a program that calculates the number of students who have a higher total score than Kristen. The program takes input from the user for the number of students (including Kristen) and their scores. The scores are stored in an array for each student. The program then compares the total score of each student with Kristen's score and determines the number of students who have a higher total score.

The program should implement the following: A class named "Student", A function named "find_larger".

The program should output the count of students and their respective total scores who have a higher total score than Kristen.

Note: The main function code is already provided. You need to implement the missing parts as described above.

Sample Input:

```
5 // Number of Students
1 2 3 4 5 // Kristen's Marks
1 2 3 4 5 // Other Student's Marks
1 2 3 4 6
1 2 3 4 4
1 2 3 4 7
```

Sample Output:

```
2 16 17 // Count of students and their respective total
```

ANSWER:

Test case 1 :

Input:
5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 6
1 2 3 4 4
1 2 3 4 7

Output:
2 16 17

Test case 2 :

Input:
5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 16
1 2 3 4 4
1 2 3 4 87

Output:
2 26 97

Test case 3 :

Input:
3
10 10 25 22 20
40 40 40 15 20
50 50 30 45 30

Output:
2 155 205

Test case 4 :

Input:
4
40 10 15 50 50
35 40 20 40 50
50 35 30 25 38
50 40 30 15 48

Output:
3 185 178 183

Test case 5 :

Input:
5
40 10 45 25 20
40 40 40 10 10
50 40 30 5 10
35 40 20 10 10
50 35 30 50 10

Output:
1 175

Test case 6 :

Input:
3
4 10 5 5 50
40 28 20 10 10

50 40 30 5 22

Output:
2 108 147

Q6. Time in seconds

Given a time in HH:MM:SS format, we have to convert the given time into total seconds using class and object approach in C++.

Sample Input:

6 //hours

30 //minutes

12 //seconds

Sample Output:

23412

Note: The body of main function is already provided in tail part.

ANSWER:

Test case 1 :

Input:
6
30
12

Output:
23412

Test case 2 :

Input:
8
12
50
Output:
29570

Test case 3 :

Input:
11
55
10
Output:
42910

Q7. Vector Class

Write a program to represent a vector in C++ using a class. The class should include the following member functions:

1. **To create the vector:** A constructor initializes the vector by taking the size and coordinates as input.
2. **To modify the vector:** A member function updates the vector coordinates with new values.
3. **To multiply the vector:** A member function multiplies all vector coordinates by a given number.
4. **To display the vector:** A member function displays the vector in the format `(10, 20, 30, ...)`.

Input Format:

1. An integer nn, the number of coordinates of the vector.
2. nn integers representing the coordinates of the vector.
3. Optionally, inputs for modifying the vector:

- nn integers as new coordinates.

1. An integer for multiplying the vector coordinates.

Output Format:

1. The vector displayed in the format `(10, 20, 30, ...)` after initialization, modification, and multiplication.

Constraints:

1. 1≤n≤1001 \leq n \leq 100 (where nn is the number of coordinates).
2. The coordinates and multiplication factor should be integers.
3. All coordinates after multiplication must fit within the range of a standard 32-bit signed integer.

Sample Input:

```
3
10 20 30
40 50 60
2
```

Sample Output:

```
Vector: (10, 20, 30)
Vector: (40, 50, 60)
Vector: (80, 100, 120)
```

Explanation:

1. The initial vector is created with 3 coordinates: 10,20,3010, 20, 30.
2. The `modify` function updates the coordinates to 40,50,6040, 50, 60.

3. The `multiply` function multiplies all coordinates by 22, resulting in 80,100,12080, 100, 120.
4. At each step, the `display` function prints the vector in the specified format.

ANSWER:

Test case 1 :

Input:
3
10 20 30
40 50 60
2

Output:
Vector: (10, 20, 30)
Vector: (40, 50, 60)
Vector: (80, 100, 120)

Test case 2 :

Input:
5
1 2 3 4 5
6 7 8 9 10
3

Output:
Vector: (1, 2, 3, 4, 5)
Vector: (6, 7, 8, 9, 10)
Vector: (18, 21, 24, 27, 30)

Test case 3 :

Input:
4
5 5 5 5
10 10 10 10
4

Output:
Vector: (5, 5, 5, 5)
Vector: (10, 10, 10, 10)
Vector: (40, 40, 40, 40)

Test case 4 :

Input:
2
0 0
3 4
5

Output:

Vector: (0, 0)
Vector: (3, 4)
Vector: (15, 20)

Test case 5 :

Input:
3
-10 -20 -30
10 20 30
2

Output:
Vector: (-10, -20, -30)
Vector: (10, 20, 30)
Vector: (20, 40, 60)

Test case 6 :

Input:
6
1 2 3 4 5 6
6 5 4 3 2 1
1

Output:
Vector: (1, 2, 3, 4, 5, 6)
Vector: (6, 5, 4, 3, 2, 1)
Vector: (6, 5, 4, 3, 2, 1)